.cell-output-stdout code { word-break: break-wor !important; white-space: pre-wrap !important; } </
style>

# Cleaning and structuring information in textual data

**Lino Galiana**
Version of October 20, 2026

As text corpora are very large objects with a low signal-to-noise ratio where the signal-to-noise ratio is low, it is necessary to carry out a series of a series of text cleaning steps. This chapter will explore some classic cleaning methods based on literary corpora literary corpora: Alexandre Dumas' *Comte de Monte Cristo* and Anglo-Saxon fantasy writers (Lovecraft, Poe, Shelley).

⚠️ **Automatically generated PDF.** This document is produced from the course website and may contain formatting issues. For an up-to-date version adapted to your reading, visit the course page: pythonds.linogaliana.fr.

## Table of contents

**1 Introduction** ..... **2**
1.1 Recap ..... 2
1.2 Chapter Objective ..... 3
1.3 Method ..... 3
**2 Example Dataset** ..... **4**
2.1 *The Count of Monte Cristo* ..... 4
2.2 The Anglo-Saxon Corpus ..... 4
**3 Initial Frequency Analysis** ..... **6**
3.1 Targeted Exploration ..... 6
3.2 Converting Text into *Tokens* ..... 8
3.3 Word Cloud: A First Generalized Analysis ..... 9
3.4 Aside: Zipf's Law ..... 15
**4 Text Cleaning** ..... **18**
4.1 Removing *Stop Words* ..... 18
4.2 Stemming and Lemmatization ..... 22
4.3 Limitation ..... 23
Informations additionnelles ..... 24
**Bibliography** ..... **25**

# 1 Introduction

## 1.1 Recap

As mentioned in the introduction to this section on natural language processing, the main goal of the techniques we will explore is the synthetic representation of language.

Natural Language Processing (NLP) aims to extract information from text through statistical content analysis. This definition includes a wide range of NLP applications (translation, sentiment analysis, recommendation, monitoring, etc.).

This approach involves transforming a text—understandable by humans—into a number, which is the appropriate format for a computer in the context of statistical or algorithmic approaches.

Turning textual information into numerical values suitable for statistical analysis is no easy task. Text data are **unstructured** since the sought-after information—specific to each analysis—is hidden in a large mass of information that must also be interpreted within a certain context (the same word or phrase can have different meanings depending on the context).

If that wasn't already difficult enough, there are additional challenges specific to text analysis, as this data is:

- **noisy**: spelling, typos…
- **evolving**: language changes with new words, meanings…
- **complex**: variable structures, agreements…
- **ambiguous**: synonymy, polysemy, hidden meanings…
- **language-specific**: no single set of rules applies across languages
- **high-dimensional**: infinite combinations of word sequences

## 1.2 Chapter Objective

In this chapter, we will focus on frequency-based methods within the *bag of words* paradigm. These are somewhat *old school* compared to the more sophisticated approaches we'll cover later. However, introducing them allows us to address several typical challenges of text data that remain central in modern NLP.

The main takeaway from this section is that since text data is very high-dimensional—language is a rich object—we need methods to reduce the noise in our text corpora to better capture the signal.

This part serves as an introduction, drawing on classic works of French and English literature. It will also present some key libraries that form part of the essential toolkit for *data scientists*: `NLTK` and `SpaCy`. The following chapters will then focus on language modeling.

> ℹ The SpaCy Library
>
> `NLTK` is the historical text analysis library in `Python`, dating back to the 1990s. The industrial application of NLP in the world of *data science* is more recent and owes a lot to the increased collection of unstructured data by social networks. This has led to a renewal of the NLP field, both in research and in its industrial application.
>
> The `spaCy` package is one of the tools that enabled this industrialization of NLP methods. Designed around the concept of data *pipelines*, it is much more convenient to use for a text data processing chain that involves multiple transformation steps.

## 1.3 Method

Text analysis aims to transform text into manipulable numerical data. To do this, it is necessary to define a minimal semantic unit. This textual unit can be a word, a sequence of $n$ words (an *ngram*), or even a string of characters (e.g., punctuation can be meaningful). This is called a **token**.

Various techniques (such as *clustering* or supervised classification) can then be used depending on the objective, in order to exploit the transformed information. However, text cleaning steps are essential. Otherwise, an algorithm will be unable to detect meaningful information in the infinite range of possibilities.

The following *packages* will be useful throughout this chapter:

```
!pip install pywaffle
!pip install spacy
```

```
!pip install plotnine
!pip install great_tables
!pip install wordcloud
```

# 2 Example Dataset

## 2.1 *The Count of Monte Cristo*

The example dataset is *The Count of Monte Cristo* by Alexandre Dumas. It is available for free on the http://www.gutenberg.org *(Project Gutenberg)* website, along with thousands of other public domain books.

The simplest way to retrieve it is to use the `request` package to download the text file and slightly clean it to retain only the core content of the book:

```
import requests
import re

url = "https://www.gutenberg.org/files/17989/17989-0.txt"
response = requests.get(url)
response.encoding = 'utf-8'  # Ensures proper decoding
raw = response.text

dumas = (
  raw
  .split("*** START OF THE PROJECT GUTENBERG EBOOK 17989 ***")[1]
  .split("*** END OF THE PROJECT GUTENBERG EBOOK 17989 ***")[0]
)


def clean_text(text):
    text = text.lower() # convert words to lowercase
    text = " ".join(text.split())
    return text

dumas = clean_text(dumas)

dumas[10000:10500]
```

**Line 13**

We extract the content of the book in a somewhat simplistic way

```
" mes yeux. --vous avez donc vu l'empereur aussi? --il est entré chez le maréchal pendant que
j'y étais. --et vous lui avez parlé? --c'est-à-dire que c'est lui qui m'a parlé, monsieur, dit
dantès en souriant. --et que vous a-t-il dit? --il m'a fait des questions sur le bâtiment, sur
l'époque de son départ pour marseille, sur la route qu'il avait suivie et sur la cargaison qu'il
portait. je crois que s'il eût été vide, et que j'en eusse été le maître, son intention eût été
de l'acheter; mais je lu"
```

## 2.2 The Anglo-Saxon Corpus

We will use an Anglo-Saxon corpus featuring three authors of gothic literature:

- Edgar Allan Poe, (*EAP*);
- HP Lovecraft, (*HPL*);
- Mary Wollstonecraft Shelley, (*MWS*).

The data is available in a CSV file provided on `GitHub`. The direct URL for retrieval is https://github.com/GU4243-ADS/spring2018-project1-ginnyqg/raw/master/data/spooky.csv.

Having a corpus that compares multiple authors will allow us to understand how text data cleaning facilitates comparative analysis.

We can use the following code to read and prepare this data:

```
import pandas as pd

url='https://github.com/GU4243-ADS/spring2018-project1-ginnyqg/raw/master/data/spooky.csv'
#1. Import des données
horror = pd.read_csv(url,encoding='latin-1')
#2. Majuscules aux noms des colonnes
horror.columns = horror.columns.str.capitalize()
#3. Retirer le prefixe id
horror['ID'] = horror['Id'].str.replace("id","")
horror = horror.set_index('Id')
```

```
```

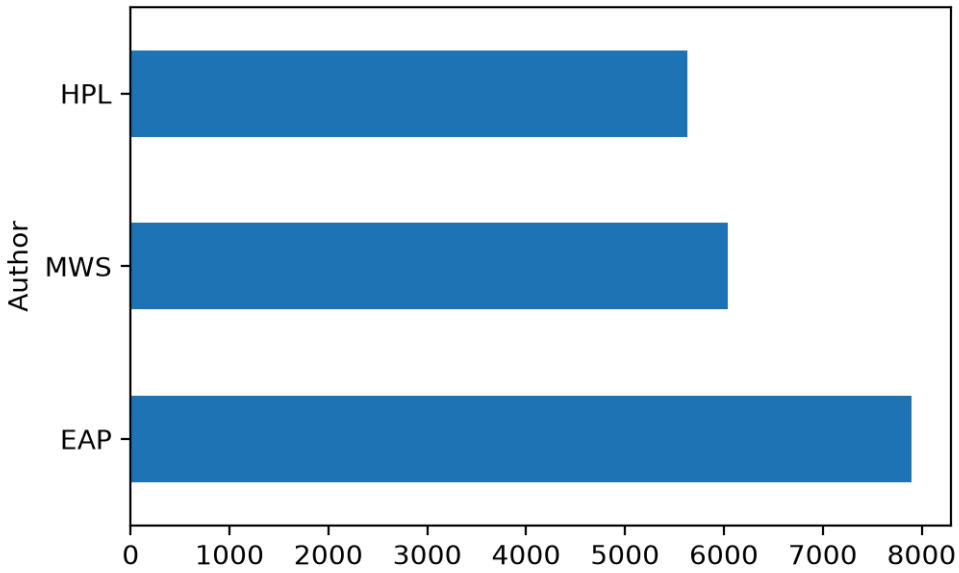
The dataset thus pairs an author with a sentence they wrote:

```
horror.head()
```

| | Text | Author | ID |
|---|---|---|---|
| Id | | | |
| id26305 | This process, however, afforded me no means of... | EAP | 26305 |
| id17569 | It never once occurred to me that the fumbling... | HPL | 17569 |
| id11008 | In his left hand was a gold snuff box, from wh... | EAP | 11008 |
| id27763 | How lovely is spring As we looked from Windsor... | MWS | 27763 |
| id12958 | Finding nothing else, not even gold, the Super... | HPL | 12958 |

We can observe that the excerpts from the 3 authors are not necessarily balanced within the dataset. If this corpus is later used for modeling, it will be important to account for this imbalance.

```
(
  horror
  .value_counts('Author')
  .plot(kind = "barh")
)
```




# 3 Initial Frequency Analysis

The standard approach in statistics—starting with descriptive analysis before modeling—also applies to text data analysis. Text mining therefore begins with a statistical analysis to determine the structure of the corpus.

Before diving into a systematic analysis of each author's lexical field, we will first focus on a single word: *fear*.

## 3.1 Targeted Exploration

Tip

The exercise below presents a graphical representation called a *waffle chart*. This is a better alternative to pie charts, which are misleading because the human eye can be easily deceived by their shape, which does not accurately represent proportions.

Exercise 1: Word Frequency

First, we will focus on our Anglo-Saxon corpus (`horror`)

1. Count the number of sentences, for each author, in which the word `fear` appears.
2. Use `pywaffle` to generate the charts below that visually summarize the number of occurrences of the word *"fear"* by author.
3. Repeat the analysis with the word *"horror"*.

The resulting count should be as follows

| | wordtoplot |
|---|---|
| Author | |
| EAP | 70 |

| | wordtoplot |
|---|---|
| Author | |
| HPL | 160 |
| MWS | 211 |

This produces the following *waffle chart*:

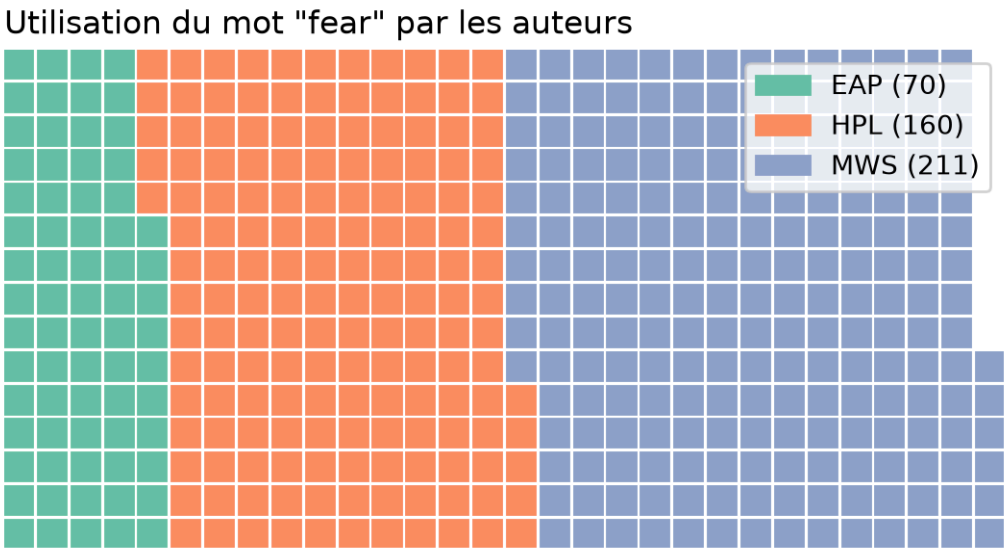



Figure 1: Distribution of the term fear in the corpus of our three authors

This clearly highlights the imbalance in our dataset when focusing on the term *"fear"*, with Mary Shelley accounting for nearly 50% of the observations.

If we repeat the analysis with the term *"horror"*, we get the following figure:

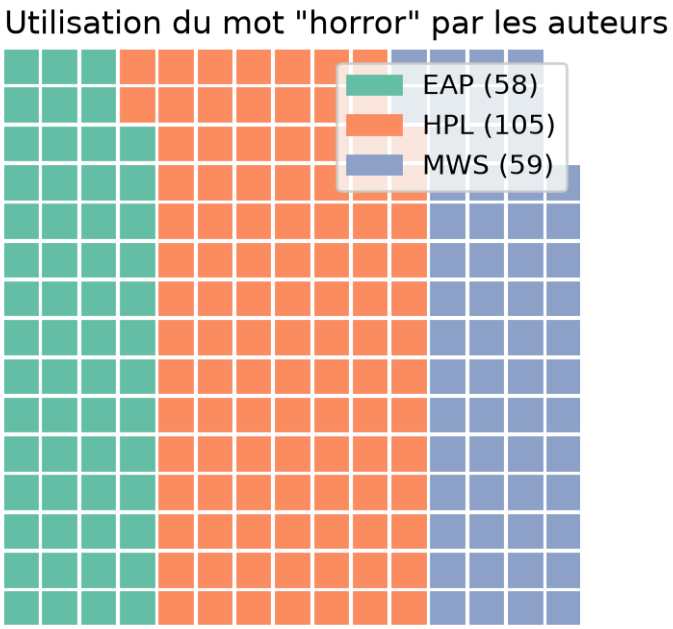



Figure 2: Distribution of the term horror in the corpus of our three authors

## 3.2 Converting Text into *Tokens*

In the previous exercise, we performed a one-off search, which doesn't scale well. To generalize this approach, a corpus is typically broken down into independent semantic units: *tokens*.

> Tip
>
> We will need to import several ready-to-use corpora to work with the `NLTK` or `SpaCy` libraries. The instructions below will help you retrieve all these resources.

To retrieve all our ready-to-use `NLTK` corpora, we do the following

```
import nltk
nltk.download('stopwords')
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('genesis')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

For `SpaCy`, it is necessary to use the following command line:

```
!python -m spacy download fr_core_news_sm
!python -m spacy download en_core_web_sm
```

Rather than implementing an inefficient *tokenizer* yourself, it is more appropriate to use one from a specialized library. Historically, the simplest choice was to use the tokenizer from `NLTK`, the classic `Python` text mining library:

```
from nltk.tokenize import word_tokenize
word_tokenize(dumas[10000:10500])
```

As we can see, this library lacks detail and has some inconsistencies: `j'y étais` is split into 4 tokens (`['j', "'", 'y', 'étais']`) whereas `l'acheter` remains a single token. `NLTK` is an English-centric library, and its tokenization algorithm is not always well-suited to French grammar rules. In such cases, it is better to use `SpaCy`, the more modern library for this kind of task. Besides being well-documented, it is better adapted to non-English languages. As shown in the documentation example on tokenizers, its algorithm provides a certain level of sophistication:

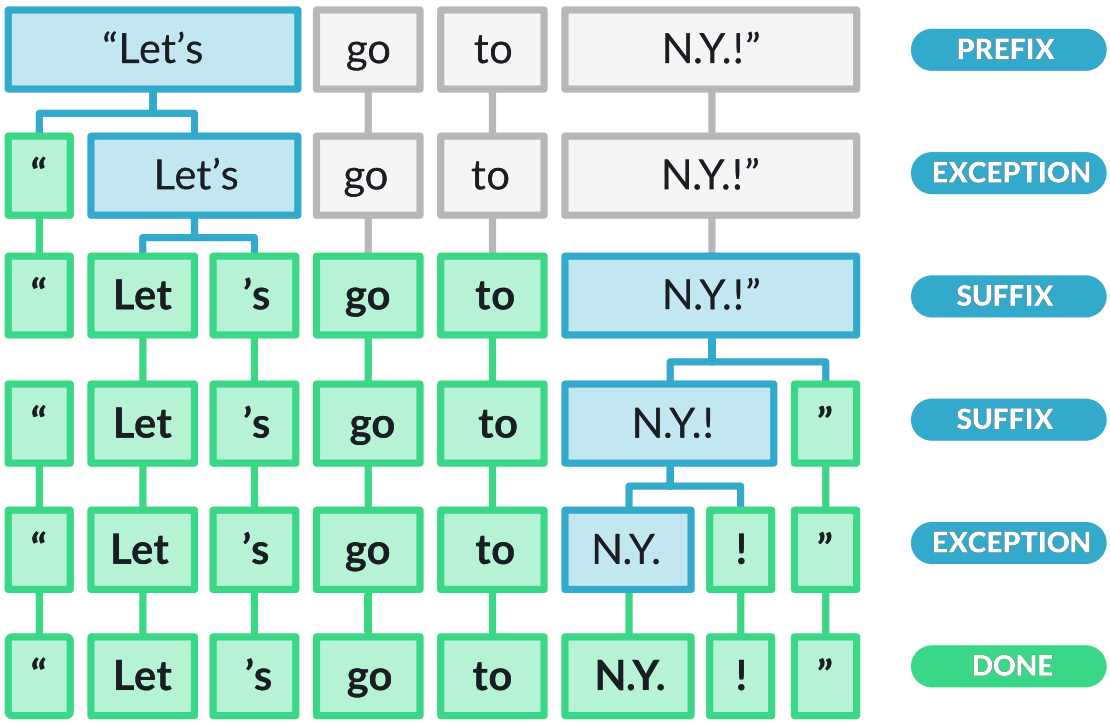



Figure 3: Example of a tokenization algorithm

It can be applied in the following way:

```
import spacy
from spacy.tokenizer import Tokenizer
nlp = spacy.load("fr_core_news_sm")
doc = nlp(dumas[10000:10500])

text_tokenized = []
for token in doc:
    text_tokenized += [token.text]

", ".join(text_tokenized)
```

```
" , mes, yeux, ., --vous, avez, donc, vu, l', empereur, aussi, ?, --il, est, entré, chez, le,
maréchal, pendant, que, j', y, étais, ., --et, vous, lui, avez, parlé, ?, --c', est, -, à, -,
dire, que, c', est, lui, qui, m', a, parlé, ,, monsieur, ,, dit, dantès, en, souriant, ., --et,
que, vous, a, -t, -il, dit, ?, --il, m', a, fait, des, questions, sur, le, bâtiment, ,, sur, l',
époque, de, son, départ, pour, marseille, ,, sur, la, route, qu', il, avait, suivie, et, sur,
la, cargaison, qu', il, portait, ., je, crois, que, s', il, eût, été, vide, ,, et, que, j', en,
eusse, été, le, maître, ,, son, intention, eût, été, de, l', acheter, ;, mais, je, lu"
```

As we can see, there are still many elements cluttering our corpus structure, starting with punctuation. However, we will be able to easily remove these later on, as we will see.

## 3.3 Word Cloud: A First Generalized Analysis

At this point, we still have no clear sense of the structure of our corpus: word count, most frequent words, etc.

To get an idea of the corpus structure, we can start by counting word distribution in Dumas’ work. Let’s begin with the first 30,000 words and count the unique words:

```
from collections import Counter

doc = nlp(dumas[:30000])

# Extract tokens, convert to lowercase and filter out punctuation and spaces
tokens = [token.text.lower() for token in doc if not token.is_punct and not token.is_space]

# Count the frequency of each token
token_counts = Counter(tokens)
```

There are already many different words in the beginning of the work.

```
len(token_counts)
```

```
1401
```

We can observe the high dimensionality of the corpus, with nearly 1,500 unique words in the first 30,000 words of Dumas' work.

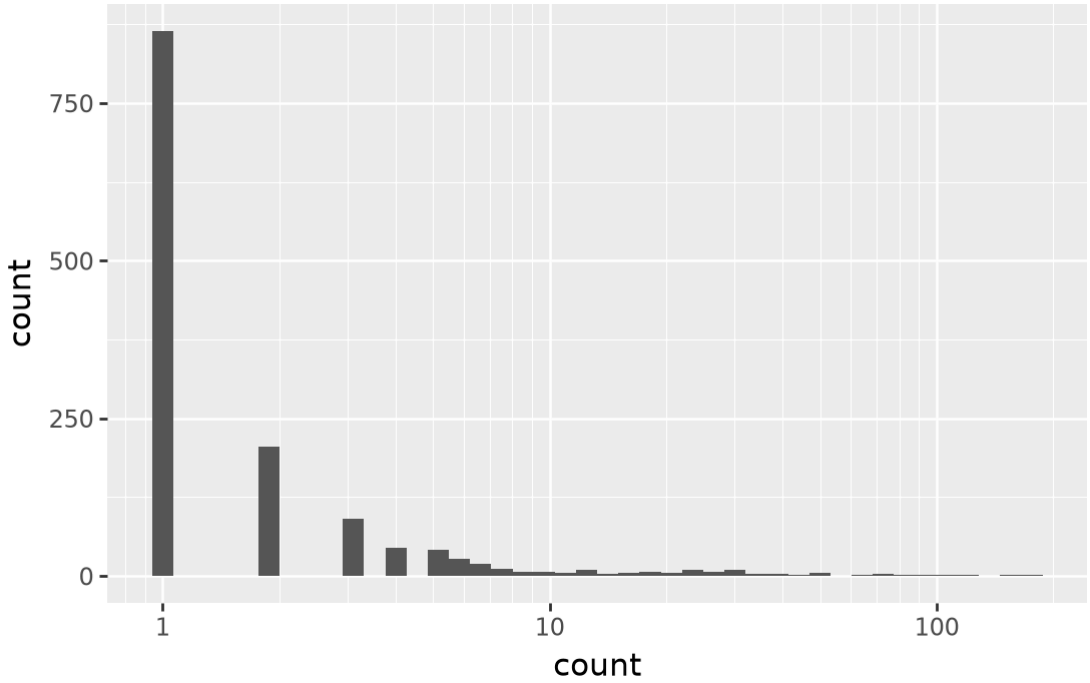
```
token_count_all = list(token_counts.items())

# Create a DataFrame from the list of tuples
token_count_all = pd.DataFrame(token_count_all, columns=['word', 'count'])
```

If we look at the distribution of word frequencies—an analysis we will extend later when discussing Zipf's law—we can see that many words are unique (nearly half), that the frequency density drops off quickly, and that we should focus more on the tail of the distribution than the following figure allows:

```
from plotnine import *
(
  ggplot(token_count_all) +
  geom_histogram(aes(x = "count")) +
  scale_x_log10()
)
```

```
/__w/python-datascientist/python-datascientist/.venv/lib/python3.13/site-packages/plotnine/
stats/stat_bin.py:112: PlotnineWarning: 'stat_bin()' using 'bins = 42'. Pick better value with
'binwidth'.
```




Now, if we look at the 25 most frequent words, we can see that they are not very informative for analyzing the meaning of our document:

```
# Sort the tokens by frequency in descending order
sorted_token_counts = token_counts.most_common(25)
sorted_token_counts = pd.DataFrame(sorted_token_counts, columns=['word', 'count'])
```

| Mot | Nombre d'occurrences | |
|---|---|---|
| de | 176 |    |
| le | 149 |    |
| et | 124 |    |
| à | 116 |    |
| l' | 102 |    |
| que | 100 |    |

*Nombre d'apparitions sur les 30 000 premiers caractères du Comte de Monte Cristo*

| Mot | Nombre d'occurrences | |
|---|---|---|
| vous | 88 |    |
| la | 83 |    |
| il | 81 |    |
| un | 77 |    |
| je | 73 |    |
| en | 70 |    |
| est | 61 |    |
| qui | 53 |    |
| dantès | 53 |    |
| d' | 50 |    |
| dit | 50 |    |
| les | 49 |    |
| du | 46 |    |
| a | 41 |    |

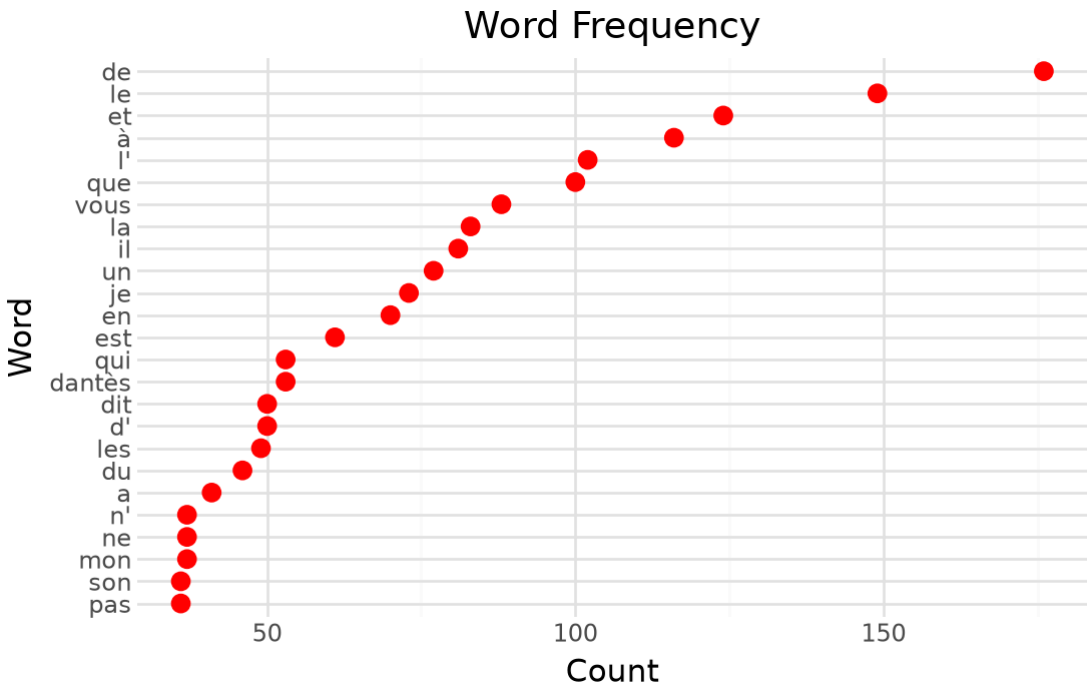
*Nombre d'apparitions sur les 30 000 premiers caractères du Comte de Monte Cristo*

| Mot | Nombre d'occurrences | |
|---|---|---|
| ne | 37 |    |
| n' | 37 |    |
| mon | 37 |    |
| son | 36 |    |
| pas | 36 |    |

*Nombre d'apparitions sur les 30 000 premiers caractères du Comte de Monte Cristo*

If we represent this ranking graphically

```
(
    ggplot(sorted_token_counts, aes(x='word', y='count')) +
    geom_point(stat='identity', size = 3, color = "red") +
    scale_x_discrete(
      limits=sorted_token_counts.sort_values("count")["word"].tolist()
    ) +
    coord_flip() +
    theme_minimal() +
    labs(title='Word Frequency', x='Word', y='Count')
)
```



We will focus on these filler words later on, as it will be important to account for them in our deeper document analyses.

Through these word counts, we've gained a first intuition about the nature of our corpus. However, a more visual approach would be helpful to gain further insight. *Word clouds* are convenient graphical representations for visualizing the most frequent words—when used appropriately. Word clouds are very easy to implement in `Python` with the `Wordcloud` module. Some formatting parameters even allow the shape of the cloud to be adjusted to match an image.

Exercise 2: Wordcloud

1. Using the `wordCloud` function, create three word clouds to represent the most commonly used words by each author in the `horror` corpus[1].
2. Create a word cloud for the `dumas` corpus using a mask like the one below.

Example mask for question 2



Figure 4: Image URL: https://minio.lab.sspcloud.fr/lgaliana/generative-art/pythonds/book.png

The word clouds generated for question 1 are as follows:

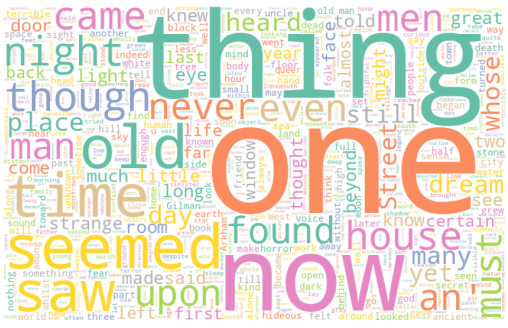



Figure 5: Lovercraft




Figure 6: Poe

[1]To obtain the same results as shown below, you can set the argument `random_state=21`.

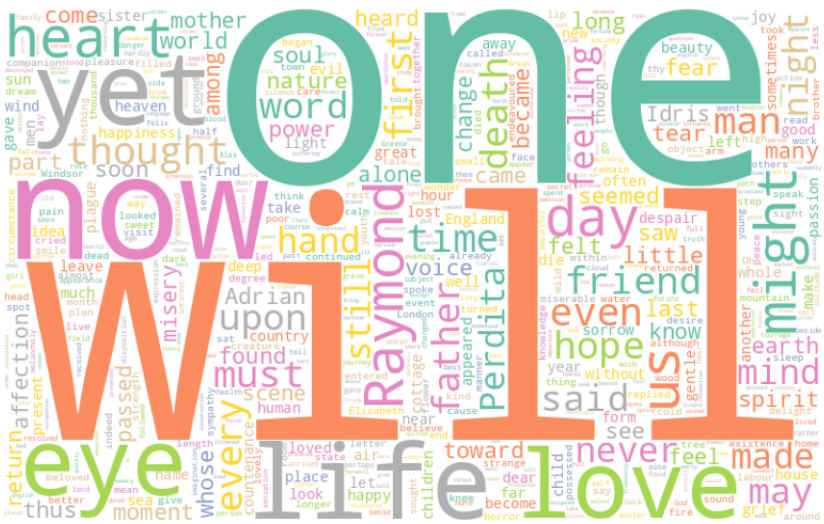



Figure 7: Shelley

Whereas the one generated from Dumas' work takes the shape

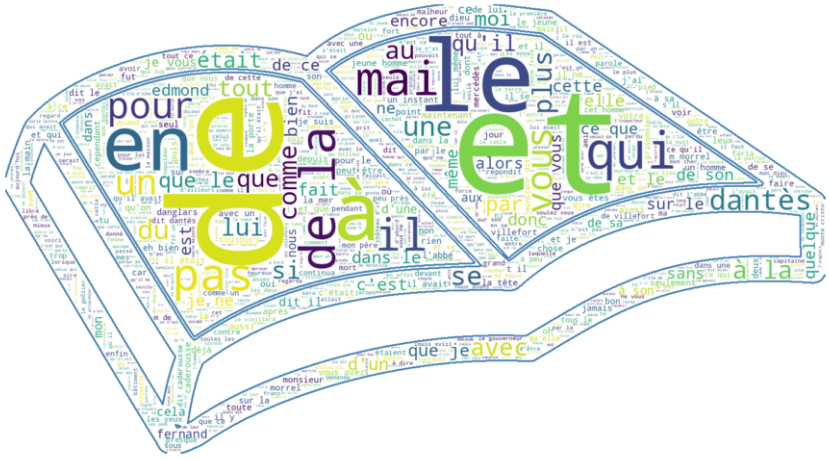



Figure 8: Word cloud produced from The Count of Monte Cristo

If it wasn't already obvious, these visualizations clearly show the need to clean our text. For instance, in the case of Dumas' work, the name of the main character, Dantès, is obscured by various articles or connecting words that interfere with the analysis. In the Anglo-Saxon corpus, similar terms like *"the"*, *"of"*, etc., dominate.

These words are called *stop words*. This is a clear example of why text should be cleaned before analysis (unless one is interested in Zipf's law, see the next exercise).

## 3.4 Aside: Zipf's Law

In the 1930s, Zipf observed a statistical regularity in Joyce's *Ulysses*. The most frequent word appeared $x$ times, the second most frequent word appeared half as often, the third a third as often, and so on. Statistically, this means that the frequency of occurrence $f(n_i)$ of a word is related to its rank $n_i$ in the frequency order by a law of the form:

$$f(n_i) = c/n_i$$

where $c$ is a constant.

More generally, Zipf's law can be derived from an exponentially decreasing frequency distribution: $f(n_i) = cn_i^{-k}$. Empirically, this means that we can use Poisson regressions to estimate the law's parameters, following the specification:

$$\mathbb{E}\Big( f(n_i)|n_i \Big) = \exp(\beta_0 + \beta_1 \log(n_i))$$

Generalized linear models (GLMs) allow us to perform this type of regression. In `Python`, they are available via the `statsmodels` package, whose outputs are heavily inspired by specialized econometric software such as `Stata`.

```
count_words = pd.DataFrame({'counter' : horror
    .groupby('Author')
    .apply(lambda s: ' '.join(s['Text']).split())
    .apply(lambda s: Counter(s))
    .apply(lambda s: s.most_common())
    .explode()}
)
count_words[['word','count']]          =          pd.DataFrame(count_words['counter'].tolist(),
index=count_words.index)
count_words = count_words.reset_index()

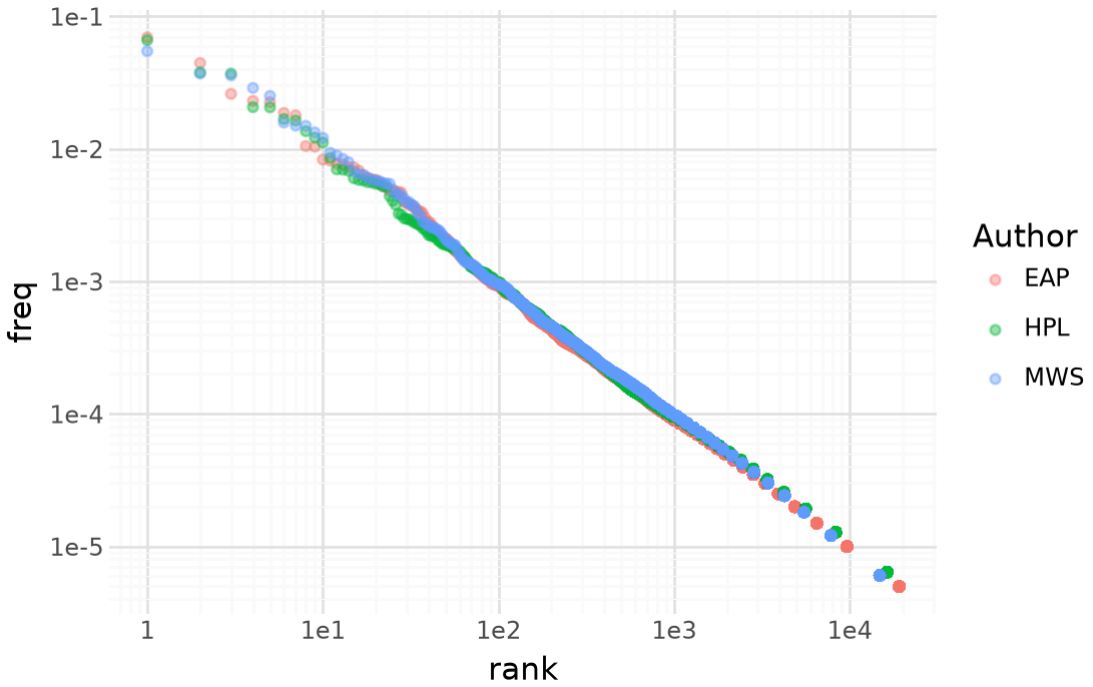
count_words = count_words.assign(
    tot_mots_auteur = lambda x: (x.groupby("Author")['count'].transform('sum')),
    freq = lambda x: x['count'] /  x['tot_mots_auteur'],
    rank = lambda x: x.groupby("Author")['count'].transform('rank', ascending = False)
)
```

Let's begin by visualizing the relationship between frequency and rank:

```
from plotnine import *

g = (
    ggplot(count_words) +
    geom_point(aes(y = "freq", x = "rank", color = 'Author'), alpha = 0.4) +
    scale_x_log10() + scale_y_log10() +
    theme_minimal()
)
```

We do indeed observe a log-linear relationship between the two in the plot:




Using `statsmodels`, let's formally verify this relationship:

```
import statsmodels.api as sm
import numpy as np

exog = sm.add_constant(np.log(count_words['rank'].astype(float)))

model = sm.GLM(count_words['freq'].astype(float), exog, family = sm.families.Poisson()).fit()

# Afficher les résultats du modèle
print(model.summary())
```

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                   freq   No. Observations:                69301
Model:                            GLM   Df Residuals:                    69299
Model Family:                 Poisson   Df Model:                            1
Link Function:                    Log   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -23.011
Date:                Tue, 20 Oct 2026   Deviance:                     0.065676
Time:                        06:44:55   Pearson chi2:                   0.0656
No. Iterations:                     5   Pseudo R-squ. (CS):          0.0002431
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const         -2.4388      1.089     -2.239      0.025      -4.574      -0.303
rank          -0.9831      0.189     -5.196      0.000      -1.354      -0.612
==============================================================================
```

The regression coefficient is close to 1, which suggests a nearly log-linear relationship between rank and word frequency. In other words, the most used word occurs twice as often as the second most frequent word, which occurs three times more than the third, and so on. This law is indeed empirically observed in this corpus of three authors.

# 4 Text Cleaning

## 4.1 Removing *Stop Words*

As we have seen, whether in French or English, a number of connecting words, while grammatically necessary, carry little informational value and prevent us from identifying the main information-bearing words in our corpus.

Therefore, it is necessary to clean our corpus by removing such terms. This cleaning process goes beyond simply removing words; it's also an opportunity to eliminate other problematic tokens, such as punctuation.

Let's start by downloading the *stopwords* corpus

```
import nltk
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to /github/home/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

```
True
```

The list of English *stopwords* in `NLTK` is as follows:

```
from nltk.corpus import stopwords
", ".join(stopwords.words("english"))
```

```
"a, about, above, after, again, against, ain, all, am, an, and, any, are, aren, aren't, as,
at, be, because, been, before, being, below, between, both, but, by, can, couldn, couldn't,
d, did, didn, didn't, do, does, doesn, doesn't, doing, don, don't, down, during, each, few,
for, from, further, had, hadn, hadn't, has, hasn, hasn't, have, haven, haven't, having, he,
he'd, he'll, her, here, hers, herself, he's, him, himself, his, how, i, i'd, if, i'll, i'm,
in, into, is, isn, isn't, it, it'd, it'll, it's, its, itself, i've, just, ll, m, ma, me,
mightn, mightn't, more, most, mustn, mustn't, my, myself, needn, needn't, no, nor, not, now, o,
of, off, on, once, only, or, other, our, ours, ourselves, out, over, own, re, s, same, shan,
shan't, she, she'd, she'll, she's, should, shouldn, shouldn't, should've, so, some, such, t,
than, that, that'll, the, their, theirs, them, themselves, then, there, these, they, they'd,
they'll, they're, they've, this, those, through, to, too, under, until, up, ve, very, was, wasn,
wasn't, we, we'd, we'll, we're, were, weren, weren't, we've, what, when, where, which, while,
who, whom, why, will, with, won, won't, wouldn, wouldn't, y, you, you'd, you'll, your, you're,
yours, yourself, yourselves, you've"
```

The list provided by `SpaCy` is more comprehensive (we have already downloaded the `en_core_web_sm` corpus in question):

```
nlp_english = spacy.load('en_core_web_sm')
stop_words_english = nlp_english.Defaults.stop_words
", ".join(stop_words_english)
```

```
"never, hereby, that, must, why, indeed, where, 'll, seems, while, whether, beforehand, give,
someone, forty, move, myself, thereby, empty, whereas, any, no, along, part, three, together,
're, did, sixty, sometime, or, upon, so, eight, therein, still, after, when, you, being, put,
their, been, only, we, yours, doing, ’re, mine, often, who, its, could, because, several, ‘m,
```

```
were, i, get, beyond, whereby, an, various, regarding, should, through, within, under, nowhere,
'll, therefore, amongst, be, each, those, another, front, against, until, afterwards, not, fifty,
your, meanwhile, once, noone, though, for, becomes, 've, above, twenty, become, something,
hereupon, whole, four, does, except, becoming, as, thus, and, however, but, between, 'm, off,
down, hereafter, cannot, using, make, me, nor, alone, 've, onto, became, ca, ten, first, us, done,
n't, elsewhere, all, take, are, whose, due, her, my, really, will, of, twelve, seeming, 're,
ever, 'm, to, already, please, ours, amount, mostly, thereafter, moreover, hence, throughout,
there, others, was, fifteen, 's, either, latter, third, last, him, thereupon, five, used, in,
six, anyhow, none, thru, whereafter, very, seemed, too, among, enough, call, nobody, hers, yet,
somewhere, two, many, almost, least, few, whoever, the, made, since, even, perhaps, behind,
itself, into, anything, herself, have, side, serious, it, other, 've, here, latterly, per, same,
from, had, bottom, thence, before, am, both, she, sometimes, everything, toward, quite, always,
just, wherein, without, neither, keep, this, every, 'd, else, with, say, everyone, go, wherever,
is, what, our, show, ourselves, out, everywhere, 'll, nine, yourself, own, whereupon, much, most,
besides, across, than, eleven, about, then, himself, on, herein, n't, anyone, name, anyway, one,
namely, more, they, well, 'd, somehow, them, has, formerly, n't, how, via, 's, below, rather,
otherwise, whenever, full, by, hundred, also, over, seem, further, 's, do, he, top, again,
which, see, whence, themselves, former, his, back, around, can, less, may, nothing, these, up,
anywhere, some, whom, towards, although, a, whither, if, such, beside, might, 'd, next, whatever,
nevertheless, at, unless, re, during, yourselves, would, now"
```

This time, if we look at the list of French *stopwords* in `NLTK`:

```
", ".join(stopwords.words("french"))
```

```
'au, aux, avec, ce, ces, dans, de, des, du, elle, en, et, eux, il, ils, je, la, le, les, leur,
lui, ma, mais, me, même, mes, moi, mon, ne, nos, notre, nous, on, ou, par, pas, pour, qu, que,
qui, sa, se, ses, son, sur, ta, te, tes, toi, ton, tu, un, une, vos, votre, vous, c, d, j, l, à,
m, n, s, t, y, été, étée, étées, étés, étant, étante, étants, étantes, suis, es, est, sommes,
êtes, sont, serai, seras, sera, serons, serez, seront, serais, serait, serions, seriez, seraient,
étais, était, étions, étiez, étaient, fus, fut, fûmes, fûtes, furent, sois, soit, soyons, soyez,
soient, fusse, fusses, fût, fussions, fussiez, fussent, ayant, ayante, ayantes, ayants, eu, eue,
eues, eus, ai, as, avons, avez, ont, aurai, auras, aura, aurons, aurez, auront, aurais, aurait,
aurions, auriez, auraient, avais, avait, avions, aviez, avaient, eut, eûmes, eûtes, eurent, aie,
aies, ait, ayons, ayez, aient, eusse, eusses, eût, eussions, eussiez, eussent'
```

We can see that this list is not very extensive and could benefit from being more complete. The one from `SpaCy` is more in line with what we would expect.

```
stop_words_french = nlp.Defaults.stop_words
", ".join(stop_words_french)
```

```
"dix, voici, ouvert, quelconque, laisser, autre, malgré, suivant, lors, elle-même, façon,
cinquante, s', chez, retour, nous-mêmes, vais, va, devant, pendant, siens, lui-même, plusieurs,
desormais, déjà, gens, dix-neuf, quatre-vingt, soi-meme, cinq, es, suivantes, nôtres, seraient,
également, differents, quatrièmement, o, maintenant, aura, tant, vas, meme, t', plutôt, celles-
la, certes, tel, rend, lès, or, de, n', pourrais, importe, entre, ouverte, qu', dix-huit, votre,
devra, relativement, quoique, nôtre, plutot, cinquantième, deuxième, doivent, sous, na, peuvent,
pouvait, tente, tres, laquelle, vôtre, telle, effet, rendre, dehors, different, semblable,
anterieures, tels, etc, vos, celle, moi, nouveau, i, ouias, seule, différent, ils, huitième,
hé, lesquels, semble, hem, nos, plus, différentes, telles, cela, parce, dès, dedans, mienne,
ouverts, anterieure, hue, l', tous, là, durant, près, ai, pas, deuxièmement, faisaient, ayant,
mien, celle-là, ah, dix-sept, selon, apres, tenant, chacun, étais, derrière, precisement, jusque,
puisque, nul, memes, cent, sait, septième, parler, elles-memes, deux, hors, fait, hui, parle,
environ, précisement, du, bat, differente, enfin, concernant, ton, quatrième, lequel, moins,
pense, souvent, suivants, delà, vous, m', as, ès, onzième, etais, l', premièrement, avec, comme,
```

```
soit, soixante, restant, dire, sera, merci, vé, allons, celles, sien, mais, me, autres, moi-
meme, se, encore, peut, directe, c', ci, à, celui-la, avais, déja, quelles, longtemps, â,
vous-mêmes, tend, nombreux, miennes, différents, suis, suit, ha, c', facon, exactement, aupres,
revoilà, serait, auxquelles, elle-meme, da, chaque, tienne, dit, cinquième, compris, suivre,
ni, celles-là, houp, seuls, etaient, mes, ce, abord, celles-ci, desquelles, ceci, et, être,
revoici, ait, peux, vers, quelle, lesquelles, quarante, seul, autrui, neanmoins, ainsi, est,
d', qui, voila, étaient, reste, sept, ont, toi-même, vu, mêmes, sinon, eux-mêmes, six, tiennes,
premier, avons, mon, tout, seize, quel, son, allaient, maint, permet, néanmoins, autrement,
spécifiques, votres, quant-à-soi, ses, pu, aie, antérieure, aurait, il, mille, treize, diverses,
envers, basee, sa, les, afin, cette, elles-mêmes, ta, ça, première, eux, eh, si, seront, très,
ouste, trente, quatorze, dits, pourquoi, étant, leur, notamment, auquel, ho, spécifique, puis,
etant, combien, un, antérieur, possible, celui-là, lui, moindres, tien, celui, partant, huit,
neuvième, aussi, quatre, stop, doit, parlent, dont, hep, vôtres, assez, antérieures, m', derriere,
tenir, anterieur, devers, deja, attendu, préalable, té, désormais, au, debout, auront, miens,
était, douzième, personne, j', semblent, sienne, troisièmement, trois, excepté, donc, notre,
aux, toi-meme, toujours, toi, directement, ces, bas, tellement, certaines, tiens, touchant, t',
nombreuses, je, hou, quant, suffisante, quiconque, outre, sont, elle, même, depuis, cet, jusqu,
uns, quand, suffisant, pourrait, desquels, ceux, par, comment, certaine, celle-la, dans, voilà,
restent, n', parfois, auraient, diverse, duquel, sur, juste, des, sauf, surtout, differentes,
on, faisant, troisième, une, moi-même, ne, toute, j', divers, semblaient, tes, eu, feront, pour,
ceux-ci, douze, relative, certains, certain, dite, egalement, parmi, via, ma, prealable, seules,
specifiques, où, fais, sent, sixième, auxquels, avaient, suivante, quinze, d', lui-meme, siennes,
avait, peu, toutes, onze, lorsque, en, leurs, celui-ci, dessus, dessous, après, seulement, unes,
etait, procedant, suffit, pres, car, quelque, nous, hormis, s', la, cependant, soi-même, ô,
revoila, quelqu'un, quelques, différente, ou, soi, possibles, proche, sans, te, font, etre, qu',
avoir, malgre, cinquantaine, specifique, tu, chacune, elles, a, vont, hi, y, quels, le, vingt,
alors, avant, celle-ci, que, dixième, déjà, ceux-là, quoi"
```

**Exercise 3: Text Cleaning**

1. Take Dumas' work and clean it using `Spacy`. Generate the word cloud again and draw your conclusions.
2. Perform the same task on the Anglo-Saxon dataset. Ideally, you should be able to use the `SpaCy` *pipeline* functionality.

```python
# Function to clean the text
def clean_text(doc):
    # Tokenize, remove stop words and punctuation, and lemmatize
    cleaned_tokens = [
        token.lemma_ for token in doc if not token.is_stop and not token.is_punct
    ]
    # Join tokens back into a single string
    cleaned_text = " ".join(cleaned_tokens)
    return cleaned_text
```

These preprocessing steps are starting to pay off since words with more meaning are starting to emerge, notably the names of the characters (Dantès, Danglars, etc.):

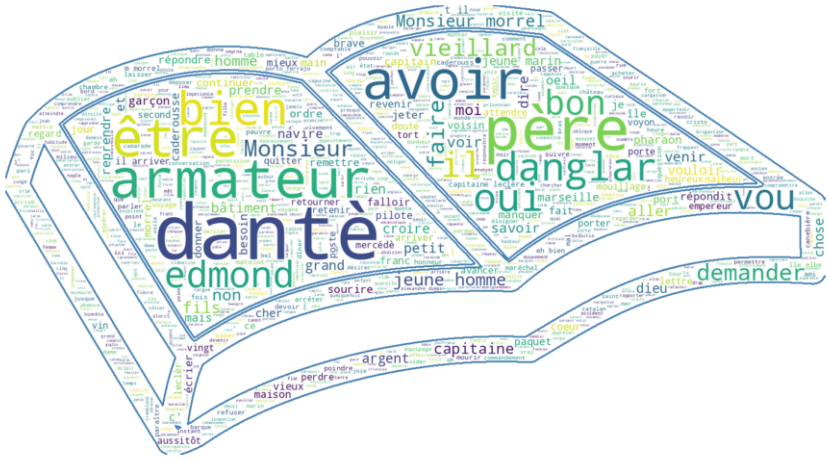



Figure 9: Word cloud produced from The Count of Monte Cristo after cleaning




Figure 10: Lovercraft




Figure 11: Poe

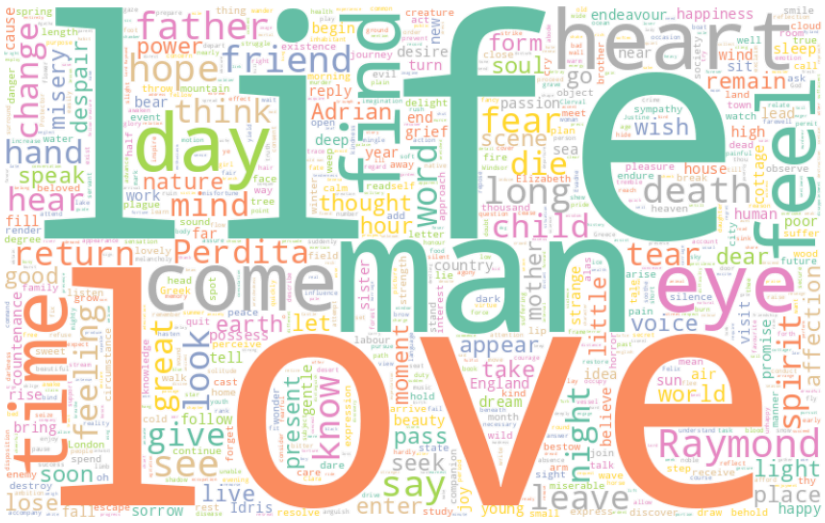



Figure 12: Shelley

## 4.2 Stemming and Lemmatization

To go further in text harmonization, it is possible to establish equivalence classes between words. For example, when conducting frequency analysis, one might want to treat *"cheval"* and *"chevaux"* as equivalent. Depending on the context, different forms of the same word (plural, singular, conjugated) can be treated as equivalent and replaced with a canonical form.

There are two main approaches:

- **Lemmatization**, which requires knowledge of grammatical roles (example: *"chevaux"* becomes *"cheval"*);
- **Stemming**, which is more rudimentary but faster, especially when dealing with spelling errors. In this case, *"chevaux"* might become *"chev"*, but that could also match *"chevet"* or *"cheveux"*.

This approach has the advantage of reducing the vocabulary size that both the computer and modeler must handle. Several stemming algorithms exist, including the *Porter Stemming Algorithm* and the *Snowball Stemming Algorithm*.

> **i** Note

To access the necessary corpus for lemmatization, you need to download it the first time using the following commands:

```
import nltk
nltk.download('wordnet')
nltk.download('omw-1.4')
```

Let's take this character string:

```
"naples. comme d'habitude, un pilote côtier partit aussitôt du port, rasa le château d'if, et
alla aborder le navire entre le cap de morgion et l'île de rion. aussitôt, co"
```

The stemmed version is as follows:

```
"napl,.,comm,d'habitud,,,un,pilot,côti,part,aussitôt,du,port,,,ras,le,château,d'if,,,et,alla,abord,le,navir,entre,le,
```

At this stage, the words become less intelligible for humans but can still be understandable for machines. This choice is not trivial, and its relevance depends on the specific use case.

Lemmatizers allow for more nuanced harmonization. They rely on knowledge bases, such as *WordNet*, an open lexical database. For instance, the words *"women"*, *"daughters"*, and *"leaves"* will be lemmatized as follows:

```
from nltk.stem import WordNetLemmatizer
lemm = WordNetLemmatizer()

for word in ["women","daughters", "leaves"]:
    print(f"The lemmatized form of {word} is: {lemm.lemmatize(word)}")
```

```
The lemmatized form of women is: woman
The lemmatized form of daughters is: daughter
The lemmatized form of leaves is: leaf
```

Exercise 4: Lemmatization with nltk

Following the previous example, use a `WordNetLemmatizer` on the `dumas[1030:1200]` corpus and observe the result.

The lemmatized version of this small excerpt from Dumas' work is as follows:

```
"naples, ., comme, d'habitude, ,, un, pilote, côtier, partit, aussitôt, du, port, ,, rasa, le,
château, d'if, ,, et, alla, aborder, le, navire, entre, le, cap, de, morgion, et, l'île, de,
rion, ., aussitôt, ,, co"
```

## 4.3 Limitation

In frequency-based approaches, where the goal is to find similarity between texts based on term co-occurrence, the question of forming equivalence classes is fundamental. Words are either identical or different—there is no subtle gradation. For instance, one must decide whether *"python"* and *"pythons"* are equivalent or not, without any nuance distinguishing *"pythons"*, *"anaconda"*, or *"table"* from *"python"*. Modern approaches, which no longer rely solely on word frequency, allow for more nuance in synthesizing the information present in textual data.

# Informations additionnelles

**i** Python environment

This site was built automatically through a `Github` action using the `Quarto` reproducible publishing software (version 1.10.18).

The environment used to obtain the results is reproducible via uv. The `pyproject.toml` file used to build this environment is available on the `linogaliana/python-datascientist` repository

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "⩾3.13,<3.14"
dependencies = [
    "altair⩾6.0.0",
    "cartiflette⩾0.2",
    "contextily==1.6.2",
    "duckdb⩾0.10.1",
    "folium⩾0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables⩾0.12.0",
    "gt-extras⩾0.0.8",
    "ipykernel⩾6.29.5",
    "jupyter⩾1.1.1",
    "jupyter-cache⩾1.0.0",
    "kaleido⩾0.2.1",
    "langchain-community⩾0.3.27",
    "loguru==0.7.3",
    "markdown⩾3.8",
    "nbclient⩾0.10.0",
    "nbformat⩾5.10.4",
    "nltk⩾3.9.1",
    "openpyxl⩾3.1.5",
    "pandas⩾3.0",
    "pip⩾25.1.1",
    "plotly⩾6.1.2",
    "plotnine⩾0.15",
    "polars⩾1.8.2",
    "py7zr⩾1.1.3",
    "pyarrow⩾17.0.0",
    "pynsee⩾0.1.8",
    "python-dotenv⩾1.0.1",
    "python-frontmatter⩾1.1.0",
    "pywaffle⩾1.1.1",
    "requests⩾2.32.3",
    "scikit-image⩾0.24.0",
    "scikit-learn⩾1.8.0",
    "scipy⩾1.13.0",
    "seaborn⩾0.13.2",
    "selenium<4.39.0",
    "spacy⩾3.8.4",
    "webdriver-manager⩾4.0.2",
    "wordcloud==1.9.3",
```

```
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

To use exactly the same environment (version of `Python` and *packages*), please refer to the documentation for `uv`.

# Bibliography